

Frontend Handoff: Doctor + Patient App (Final)

1) Project Goal

This document maps each frontend page to backend APIs, request shapes, expected response data, and integrations.

Use this as the single source for frontend binding.

2) Base Config

- Base URL: ``<BASE_URL>``
- Local example: ``http://localhost:8080``
- Auth header for protected APIs: ``Authorization: Bearer <JWT_TOKEN>``
- Content-Type: ``application/json``

3) Roles and Access

- ``DOCTOR``: diagnosis + prescription write, dashboard, read patient data, billing access.
- ``PATIENT``: read-only endpoints for own app pages.
- ``NURSE``: not main focus here, but some shared reads exist.

4) Doctor App Pages and Features

Page: Login

Feature: doctor signs in and receives JWT.

- ``POST /auth/login``

Request:

```
```json
{
 "username": "doctor1",
 "password": "pass123"
}
```
```

Response (important fields):

- ``token``
- ``tokenType``
- ``username``
- ``role``

Page: Doctor Dashboard

Feature: quick hospital summary and recent activity.

- ``GET /doctor/dashboard``

Expected keys in response:

- ``totalPatients``
- ``recentPatients``
- ``healthSnapshot``
- ``activity``

Page: Patient List

Feature: list/search/filter patients.

- ``GET /patients``
- ``GET /patients?search=&status=&page=&limit=`
- ``GET /patients/search?query=`

Page: Patient Detail

Feature: full profile + timeline blocks.

- ``GET /patients/{id}``
- ``GET /patients/{id}/history`
- ``GET /patients/{id}/vitals/current`

- `GET /patients/{id}/diagnoses`
- `GET /patients/{id}/prescriptions`

Page: Add/Edit Patient

Feature: doctor can create and update patient profile.

- `POST /patients`
- `PUT /patients/{id}`
- `PATCH /patients/{id}`
- `DELETE /patients/{id}` (only when UI confirms)

Page: Diagnosis Management

Feature: create/list/filter/update/delete diagnosis.

- `POST /diagnoses`
- `GET /diagnoses/patient/{patientId}`
- `GET /diagnoses?patientId=&priority=&status=`
- `PUT /diagnoses/{id}`
- `PATCH /diagnoses/{id}/status`
- `DELETE /diagnoses/{id}`

Page: Prescription Management

Feature: create/list/filter/update/delete prescriptions and download PDF.

- `POST /prescriptions`
- `GET /prescriptions/{id}`
- `GET /prescriptions/patient/{patientId}`
- `GET /prescriptions?patientId=&status=`
- `PUT /prescriptions/{id}`
- `PATCH /prescriptions/{id}/status`
- `DELETE /prescriptions/{id}`
- `GET /prescriptions/{id}/pdf`

Page: Queue Read

Feature: doctor can view queue.

- `GET /queue`

Page: Billing (Doctor side)

Feature: invoice lifecycle.

- `GET /billing/patients/search?q=`
- `GET /insurance/providers`
- `POST /billing/invoices`
- `GET /billing/invoices/{id}`
- `PUT /billing/invoices/{id}`
- `POST /billing/invoices/{id}/items`
- `PUT /billing/invoices/{id}/items/{itemId}`
- `DELETE /billing/invoices/{id}/items/{itemId}`
- `POST /billing/invoices/{id}/draft`
- `POST /billing/invoices/{id}/send`
- `GET /billing/invoices/{id}/pdf`

5) Patient App Pages and Features

Page: Register

- `POST /auth/register`

Page: Login

- `POST /auth/login`

Page: Profile (top bar/user section)

- `GET /auth/me`

Page: My Profile

- `GET /patients/{id}`

Page: Medical History

- `GET /patients/{id}/history`

Page: My Diagnoses

- `GET /diagnoses/patient/{patientId}`

- `GET /patients/{id}/diagnoses`

Page: My Prescriptions

- `GET /prescriptions/patient/{patientId}`

- `GET /patients/{id}/prescriptions`

- `GET /prescriptions/{id}`

- `GET /prescriptions/{id}/pdf`

Page: My Vitals

- `GET /vitals/patient/{patientId}`

- `GET /patients/{id}/vitals/current`

6) Request Payloads (Most Used)

Patient Create/Update

```json

```
{
 "firstName": "Ravi",
 "lastName": "Kumar",
 "gender": "MALE",
 "dateOfBirth": "1998-04-21",
 "phone": "+919999999999",
 "email": "ravi@mail.com",
 "address": "Bangalore",
 "status": "ACTIVE",
 "wardId": "WARD-1",
 "priority": "NORMAL"
}
```

```

Diagnosis Create/Update

```json

```
{
 "patientId": 1,
 "diagnosisDate": "2026-04-30",
 "conditionName": "Viral Fever",
 "notes": "Take rest",
 "doctorName": "Dr. Sharma",
 "priority": "HIGH",
 "status": "OPEN"
}
```

```

Prescription Create/Update

```json

```
{
 "patientId": 1,
 "prescribedDate": "2026-04-30",
 "instructions": "After food",
 "doctorName": "Dr. Sharma",
 "status": "ACTIVE",
}
```

```

"medicines": [
 {
 "medicineName": "Paracetamol",
 "dosage": "500mg",
 "frequency": "2 times/day",
 "durationDays": 5
 }
]
}
```

```

```

### Status Patch
```json
{
 "status": "ACTIVE"
}
```

```

7) Frontend Integration Rules

1. Store JWT after login and attach it to every protected request.
2. Use `/diagnoses` as main path (plural). `/diagnosis` is legacy-compatible.
3. Do not call `/prescriptions/{id}/history` because it does not exist.
4. PDF endpoints return file bytes, not JSON.
5. Date formats:
 - `LocalDate`: `YYYY-MM-DD`
 - `LocalDateTime`: `YYYY-MM-DDTHH:mm:ss`

8) Standard Error Response

```

```json
{
 "timestamp": "2026-04-30T03:48:08.752",
 "status": 400,
 "error": "Bad Request",
 "message": "Validation failed",
 "details": [
 "fieldName: reason"
]
}
```

```

Frontend should show:

- `message` as primary error.
- `details[]` inline with form fields when present.

9) Quick API Smoke Checklist Before Binding

1. `POST /auth/login` -> token received.
2. `GET /auth/me` with bearer token -> username and role.
3. `GET /doctor/dashboard` with doctor token.
4. `GET /patients` with doctor token.
5. `GET /patients/{id}/history` with doctor token.
6. `GET /diagnoses/patient/{id}` with doctor token.
7. `GET /prescriptions/patient/{id}` with doctor token.
8. `GET /prescriptions/{id}/pdf` with doctor/patient token.

If all above pass, frontend binding can start safely.

